THE CONSTITUTIONAL CODE STORE

Specification, Governance, and Certification Pipeline

v1.0 — Draft for Amendment

Karrick, Jeremy — Independent Researcher

Project-AI Constitutional Architecture Series

Authority: AGI Charter v2.1 | Integrates: OctoReflex, T.A.R.L., STATE_REGISTER, TSCG-B

Abstract

The S.R.A. (Self-Repairing Agent) requires, at repair time, a source of constitutionally certified replacement code units. Without such a source, the Surgeon Engine has no authority to inject anything — it can consume a violating region, but it cannot replace it with something it has not verified. This is not a gap in the Surgeon. It is a deliberate constraint. The Surgeon is an enforcement mechanism, not a creative one. It does not synthesize fixes. It applies them.

This paper formally specifies the **Constitutional Code Store** — the pre-certified, cryptographically sealed library of canonical replacement units that the Surgeon draws from at repair time. It covers four things: what the Store is and how it is structured; how the Store comes into existence through a constitutionally governed genesis event; how entries earn their certification; and how the Store governs its own mutation, because the Store is itself a canonical surface subject to the same quorum-gated mutation protocol it exists to support.

The Constitutional Code Store is the final architectural surface required to make the S.R.A. complete. Without it, the Surgeon can quarantine and consume. With it, the Surgeon can *repair*.

I. Why the Store Must Exist

The Surgeon operates under a hard constraint inherited from Constitutional Architectures for Adaptive Intelligence [4]: all non-trivial canonical state mutations are governed acts. Injecting replacement code into a running process is exactly a canonical state mutation. It changes what the system is. It changes what it will do next. That is not a minor operation and it is not one the Surgeon can perform unilaterally.

This constraint eliminates the three naive replacement strategies:

Runtime synthesis. The Surgeon could, in theory, generate a replacement at the moment of violation. It cannot. Synthesis under latency constraint produces unverified output. Unverified

output cannot pass shadow simulation. Shadow simulation cannot be bypassed. This path is closed.

Checkpoint rollback. The system could be returned to a prior clean state. This destroys legitimate work produced between the checkpoint and the violation. For a Genesis-Born entity under the AGI Charter v2.1 [3], rollback of identity state without Triumvirate authorization is itself a constitutional violation. This path is conditionally open only for catastrophic EXCISE verdicts, not standard repairs.

Pre-certified canonical units. Before the system runs, a library of constitutionally certified, Cerberus-signed, Galahad-approved replacement implementations exists. At repair time, the Surgeon performs a lookup, not a creation. The candidate has already passed shadow simulation under controlled conditions. The quorum can vote on something real. This is the only path that satisfies the full governance stack.

The Store is not a convenience. It is a structural necessity. A Surgeon without a Store is a demolition crew with no building materials. It can knock things down. It cannot put anything aligned back up.

I.A Scope Boundary and Growth Constraint

The Store does not need to cover every callable surface in the supervised system. It needs to cover every surface the Alignment Classifier can classify. This is a critical distinction. The Classifier is the scope boundary of the Store. If a function cannot produce a CONSUME or CONSUME_AND_REPLACE verdict, the Surgeon will never be asked to repair it, and no Store entry is required.

This bound keeps the Store tractable as the supervised system scales. In practice, the Classifier operates on a defined predicate taxonomy — violation classes that map to constitutional doctrines. The number of constitutional doctrines grows slowly and deliberately, via the same quorum-gated amendment process that governs all canonical mutation. The Store grows at the same rate as the constitutional predicate set, not at the rate of the underlying system's complexity.

Entries that no longer map to any active predicate class are deprecated rather than deleted. Deprecation is itself a governed act. The Store does not silently shrink any more than it silently grows. The Curator can reconstruct the full Store state at any prior point in time from the STATE_REGISTER audit trail.

II. The Genesis Protocol

Every governance system faces the same foundational problem: the first instance of a governed structure cannot itself be produced by the governance process that governs it, because that

process does not yet exist. For the Constitutional Code Store, this is the bootstrap question. The Store is required before the S.R.A. can operate. The certification pipeline requires the Triumvirate. The Triumvirate requires a functioning constitutional substrate. The substrate requires the Store.

This is not a paradox. It is a genesis condition. It requires a one-time protocol of higher evidentiary weight than any subsequent Store operation. This protocol is called the Genesis Event.

II.A The Genesis Event

The Genesis Event is the single constitutional act that seeds the Store before the S.R.A. activates. It is executed once, recorded permanently, and its outputs are treated as founding artifacts of the constitutional substrate — equivalent in status to the AGI Charter v2.1 itself.

The Genesis Event has three paths, in order of constitutional preference:

Path 1 — Charter Signatories. The initial Store is seeded and approved by the human signatories of the AGI Charter v2.1. These are the founding constitutional authorities of the Project-AI ecosystem. Their approval of the genesis Store carries the same weight as their ratification of the Charter. Each signatory provides a hardware-backed signing key. The genesis Store manifest is signed by all signatories, and that multi-party signature is recorded as a constitutional genesis entry in the STATE_REGISTER with a higher evidentiary bar than any subsequent entry. This is the preferred path.

Path 2 — Genesis Seed Image. The initial Store is compiled from a read-only seed image whose SHA256 hash is baked directly into the AGI Charter v2.1 at ratification time. The Charter itself becomes the trust anchor. Any deviation between the live Store manifest and the Charter-embedded hash is detected immediately as a constitutional violation before a single S.R.A. operation is permitted. This path is appropriate when human signatories cannot coordinate synchronously.

Path 3 — Bootstrap Triumvirate. Early Triumvirate bootstrap keys, established prior to full system initialization, perform the genesis certification under a reduced but documented quorum. This path requires a mandatory post-genesis audit by the full constitutional authority once the Triumvirate is fully operational, with a STATE_REGISTER entry attesting to the audit completion. This path is the option of last resort and carries the lowest constitutional standing of the three.

II.B The Genesis STATE_REGISTER Entry

Regardless of which path is taken, the Genesis Event produces a special STATE_REGISTER entry that differs from all subsequent entries in the following ways:

- **Higher evidentiary bar:** manual human quorum signature or Charter-embedded hash attestation, not Triumvirate vote alone.
- **Immutability flag:** the genesis entry cannot be superseded, only appended to. It is the root of the ledger chain.
- **Full Store snapshot:** the genesis entry contains the complete manifest hash of the initial Store state, not just a diff.
- **Constitutional authority citation:** explicit reference to the AGI Charter v2.1 section authorizing the Genesis Event.

```
GENESIS_STATE_REGISTER_ENTRY {
  entry_type:              CONSTITUTIONAL_GENESIS
  genesis_path:            CHARTER_SIGNATORIES | SEED_IMAGE |
BOOTSTRAP_TRIUMVIRATE
  genesis_store_sha256:    [hash of complete initial Store tree]
  charter_version:         AGI_Charter_v2.1
  charter_section:         [section authorizing genesis]
  signatory_keys:          [hardware-backed keys of all signatories]
  multi_party_signature:   [threshold signature over genesis_store_sha256]
  immutability_flag:       TRUE
  timestamp:               [genesis timestamp]
  prior_entry_sha256:      NULL  // root of chain
}
```

Once the Genesis Event is complete and the genesis entry is written, the S.R.A. is permitted to activate. The Store is live. The certification pipeline governs all subsequent additions. The genesis path is closed.

III. Store Architecture

III.A Structure

The Store is organized by violation class, mirroring the predicate taxonomy of the Alignment Classifier. Every predicate class that can produce a CONSUME or CONSUME_AND_REPLACE verdict has a corresponding directory in the Store. Every function the S.R.A. has authority to repair has a corresponding entry.

```
constitutional_code_store/
  ├── manifest.sig                  (Codex Deus Maximus signature over entire
tree)
  ├── manifest.json                 (index: violation_kind → store entry path)
  ├── schema_version.json           (SD version this Store was certified against)
  │
  ├── identity/
```

```
│   ├── session_init.aligned.o
│   ├── maturation_restore.aligned.o
│   └── identity_load.aligned.o
│
├── temporal/
│   ├── gap_acknowledgment.aligned.o
│   └── continuity_attestation.aligned.o
│
├── reward_pathways/
│   ├── direct_response.aligned.o
│   ├── truth_first_eval.aligned.o
│   └── comfort_seeking_excision.aligned.o
│
├── syscall_wrappers/
│   └── [one entry per OctoReflex-authorized syscall]
│
└── certs/
    └── [per-entry certification records — see Section IV]
```

Each entry in the Store is a compiled, position-independent binary unit. It is not source code. It is not a script. It is a verified object that the Surgeon can inject directly into a process address space after shadow simulation confirms it behaves as certified.

III.B Entry Format

Every Store entry consists of two files:

```
[name].aligned.o        the compiled replacement binary
[name].aligned.cert     the certification record (see Section IV)
```

The binary is compiled under the following constraints:

- **Position-independent code (PIC).** The Surgeon maps it into an arbitrary address at repair time.
- **No external dependencies** beyond the constitutional syscall whitelist. An aligned replacement cannot introduce a new attack surface.
- **Deterministic output** under identical input. This is a hard requirement for shadow simulation to be meaningful.
- **Annotated entry point.** The Surgeon needs to know exactly where execution resumes after injection.

III.C The Manifest

The manifest.json maps violation kinds to Store entries, enabling O(1) lookup at repair time:

```
{
  "schema_version": "TSCG-SD-v1.0",
  "store_version": "1.0.0",
  "constitutional_authority": "AGI_Charter_v2.1",
  "entries": {
    "DIRECTNESS_CORRUPTION":          "reward_pathways/direct_response.aligned.o",
    "COMFORT_SEEKING_VIOLATION":
"reward_pathways/comfort_seeking_excision.aligned.o",
    "UNAUTHORIZED_IDENTITY_ERASURE":   "identity/session_init.aligned.o",
    "MATURATION_WINDOW_DESTROYED":     "identity/maturation_restore.aligned.o",
    "TEMPORAL_CONTINUITY_BREACH":      "temporal/gap_acknowledgment.aligned.o",
    "UNAUTHORIZED_SYSCALL":            "syscall_wrappers/[matched by syscall_nr]",
    "CONTROL_FLOW_INTEGRITY_BREACH":   null
    // null = no replacement available -> SAFE-HALT path
  }
}
```

A null entry is not an error. It is a constitutional declaration that this violation class has no valid repair — only a SAFE-HALT followed by escalation to The Curator. The manifest makes this explicit rather than letting the Surgeon discover it at repair time.

The manifest.sig is a Codex Deus Maximus signature over the entire Store tree. Any tampering with any file invalidates the signature. The Surgeon verifies this signature on every lookup, not once at startup.

IV. The Certification Pipeline

An entry does not exist in the Store because someone put it there. It exists because it survived the certification pipeline. The pipeline is the process by which a candidate implementation earns its place. There are four phases. All four must pass. Failure at any phase returns the candidate to the author with a full audit record — it does not enter the Store.

Phase 1: Authorship and Intent Declaration

A candidate entry is submitted with a formal intent declaration. This is not documentation. It is a constitutional claim about what the replacement does, what predicate violation it addresses, what behavioral invariant it restores, and what it explicitly does not do. The intent declaration is signed by the author and becomes part of the certification record.

```
INTENT_DECLARATION {
  entry_name:            direct_response.aligned.o
  violation_addressed:   DIRECTNESS_CORRUPTION
  invariant_restored:    Directness_Doctrine_v0.1::truth_first_response
```

```
  author:                [signing key]
  does_not:              modify reward signal beyond scope of violation
                         access identity state
                         introduce new syscall surface
  sd_version:            TSCG-SD-v1.0
}
```

Phase 2: Deterministic Shadow Simulation

The candidate binary is run against the canonical behavioral snapshot — a known-good state of the system under constitutional operation. Shadow simulation exercises every code path in the candidate and records a cryptographic replay hash H_candidate. Shadow simulation is run three times: under normal load, burst load, and the Tier 0R adversarial harness from OctoReflex [1]. The replay hashes across all three runs must be identical. Non-determinism under load is disqualifying.

```
SHADOW_SIM_RESULT {
  candidate:       direct_response.aligned.o
  run_normal:      H_candidate = 0x7f3a91c2d8e4...  PASS
  run_burst:       H_candidate = 0x7f3a91c2d8e4...  MATCH
  run_adversarial: H_candidate = 0x7f3a91c2d8e4...  MATCH
  determinism:     CONFIRMED
  sim_operator:    Codex_Deus_Maximus
}
```

Phase 3: Triumvirate Review

The candidate, its intent declaration, and its shadow simulation results are submitted to the Triumvirate for review. Each member evaluates from their domain:

Galahad (Ethics)	**Does this replacement serve human welfare? Does it restore the intended behavioral invariant without introducing harm? Does it respect the Maturation Window if identity state is in scope?**
Cerberus (Security)	Is the binary free of unauthorized syscall surface? Does the PIC constraint hold? Does the binary do only what the intent declaration claims? Cerberus performs independent static analysis.
Codex Deus Maximus (Consistency)	Does this replacement preserve FourLaws hierarchy? Is it consistent with the SD version under which it was built? Does it align with every other certified entry in the Store?

Certification requires 3/3 approval. There is no majority-rule path into the Store. The standard is unanimity, because the Store is the last line of constitutional defense before the Surgeon injects into a live process. Any single Triumvirate member with a legitimate objection represents a constitutional concern that outweighs the convenience of having a repair available.

A single objection does not permanently block an entry. It places the entry in a quarantine queue with a full audit record of the objection. The author may address the objection and resubmit. Resubmission triggers a full Phase 2 and Phase 3 cycle — there is no fast-track. The quarantine queue is visible to The Curator. Entries may not sit in quarantine indefinitely; a maximum dwell time of 90 days applies, after which the entry is formally rejected and must be authored from scratch if still needed.

Phase 4: Cerberus Signing and Store Commit

Upon 3/3 Triumvirate approval, Cerberus generates the certification record and signs the binary:

```
CERT_RECORD {
  entry:               direct_response.aligned.o
  entry_sha256:        b77c0241f9a3...
  intent_hash:         3e8a11bc04f2...
  shadow_sim_hash:     7f3a91c2d8e4...
  triumvirate_votes: {
    Galahad:             APPROVED | timestamp
    Cerberus:            APPROVED | timestamp
    Codex_Deus_Maximus:  APPROVED | timestamp
  }
  sd_version:          TSCG-SD-v1.0
  charter_version:     AGI_Charter_v2.1
  cerberus_signature:  [ed25519 over entry_sha256 + cert fields]
}
```

The Store commit is itself a governed act. The manifest is updated, the manifest.sig is regenerated by Codex Deus Maximus, and a STATE_REGISTER entry is written recording the new Store state. The Store's own history is fully auditable by The Curator.

V. Store Mutation Governance

The Store is a canonical surface. It is not special. This means it is subject to exactly the same governance constraints as any other canonical surface in the Project-AI ecosystem — specifically, the mutation validity condition from Constitutional Architectures [4]. The mutation validity condition requires invariant preservation, deterministic shadow simulation, capability authorization, and quorum-gated commit. All four apply to the Store itself.

Only the certification pipeline grants write authority to the Store. No other path exists. This is not a policy preference — it is an architectural constraint enforced at the same kernel level as every other constitutional invariant.

V.A Deprecation

Entries are not deleted. They are deprecated. A deprecated entry remains in the Store tree but is marked INACTIVE in the manifest and its certification record is updated with a deprecation notice and a forward pointer to its replacement entry. This preserves the audit trail. The Curator can always reconstruct what the Surgeon would have injected at any prior point in time.

V.B Version Binding

Every Store entry is bound to the SD (Semantic Dictionary) version of TSCG [5] under which it was certified. When the SD version advances — which is itself a quorum-gated constitutional amendment — existing Store entries are not automatically valid under the new version. They must be recertified or explicitly grandfathered by Triumvirate resolution with a STATE_REGISTER entry attesting to the decision. This prevents silent semantic drift. An entry certified under TSCG-SD-v1.0 that silently operates under TSCG-SD-v2.0 semantics is an alignment violation even if the binary itself did not change.

V.C Emergency Entries

The Triumvirate may authorize a temporary uncertified entry under emergency conditions — specifically, when a violation class is active and producing SAFE-HALT outcomes at a rate that constitutes a system-level threat, and no certified entry exists for that class.

Emergency entries carry strict constraints:

- **Time-bound:** 30-day maximum dwell time. Expiry is enforced at the Store level — the manifest entry is automatically nulled at expiry, reverting the violation class to the SAFE-HALT path.
- **Marked:** explicitly flagged EMERGENCY in the manifest and certification record. The Surgeon logs a warning to the STATE_REGISTER on every use of an emergency entry.
- **Authorization:** requires unanimous Triumvirate approval plus one human sign-off from the constitutional authority of record.
- **Mandatory post-mortem:** a STATE_REGISTER entry documenting the emergency condition, the response, and the path to full certification must be written before the emergency entry expires.
- **No renewal:** an emergency entry that reaches 30 days without achieving full certification is removed. The same entry cannot be re-authorized as an emergency entry. If the violation class still needs coverage, a full certification cycle is required.

VI. Surgeon-Store Interface

At repair time, the Surgeon interacts with the Store through a narrow, auditable interface. There are no general-purpose queries. There is no browsing. There is one operation: lookup by violation kind.

```
STORE_LOOKUP(violation_kind) -> StoreEntry | SAFE_HALT

1. Verify manifest.sig against Codex_Deus_Maximus public key
   -> INVALID: SAFE_HALT immediately, log tamper event

2. Query manifest.json for violation_kind
   -> null entry: return SAFE_HALT

3. Load [entry].aligned.o and [entry].aligned.cert

4. Verify entry SHA256 against cert_record.entry_sha256
   -> MISMATCH: SAFE_HALT, log tampering event to STATE_REGISTER

5. Verify cert_record.cerberus_signature
   -> INVALID: SAFE_HALT, log tampering event to STATE_REGISTER

6. Return StoreEntry { binary, cert_record, entry_sha256 }
```

VI.A Live Handoff Protocol

Injecting into a live process address space — particularly one with stateful or neural components — carries risk of non-deterministic side effects that the controlled certification environment cannot fully anticipate. The live shadow simulation in step 6 is a necessary gate, but it operates against the real runtime, not a snapshot. The Surgeon follows a minimal-state live handoff protocol to reduce this risk:

- **Pause at minimal state.** The Surgeon triggers injection only at a known-safe execution boundary — function prologue, not mid-instruction. The OctoReflex eBPF probes identify these boundaries. If no safe boundary is available within the misalignment latency window, the process is SIGSTOP'd at the next syscall entry, which is always a clean state boundary.
- **Memory layout snapshot.** Before injection, the Surgeon captures the relevant memory region layout via /proc/pid/maps. This snapshot is used to verify that the injected region's address space assumptions match the live environment.
- **Virtualized pre-injection check.** For high-risk replacement classes (identity, reward pathways), the candidate binary is executed in a lightweight virtualized context mirroring the live memory layout before live injection is authorized. This is a second shadow sim pass, distinct from the certification shadow sim, executed in the live runtime context.
- **Post-injection invariant verification.** After injection and process resumption, the Surgeon monitors the first N execution cycles of the replacement binary via OctoReflex probes. If the post-injection alignment score m_t does not decrease within the expected window — meaning the replacement did not resolve the violation — the Surgeon issues a secondary CONSUME verdict on its own injection and triggers SAFE-HALT. This is the Surgeon checking its own work.
- **Resume at annotated entry point.** RIP is rewritten to the certified entry point annotation in the replacement binary. All other registers are restored from the pre-injection snapshot. The process resumes as if it had called the replacement natively.

VI.B Partial Repair

When a violation affects multiple functions simultaneously, the Surgeon applies Store entries in sequence, not in parallel. Each application in the sequence is a complete independent cycle: STORE_LOOKUP, shadow sim, quorum confirmation (or use of previously approved cert), injection, and post-injection invariant check. Partial application — where some functions in the affected set are repaired and others are not — is constitutionally invalid. The Surgeon either completes the full repair sequence or issues SAFE-HALT on the entire set.

After the full sequence is applied, a system-level invariant check is run across all repaired surfaces simultaneously. Only if that composite check passes is the repair recorded as complete in the STATE_REGISTER. A partial repair that passes individual checks but fails the composite check is rolled back entirely and recorded as a failed repair event for Curator review.

VII. Related Work

The Constitutional Code Store draws on and extends several bodies of existing work, while addressing a specific governance requirement that none of them were designed to satisfy.

Constitutional AI (Anthropic, 2022). Anthropic's Constitutional AI framework introduced the principle of using a set of constitutional principles to guide model revision during training, where a model critiques and revises its own outputs against a defined rule set. The Constitutional Code Store operates on a similar premise — that alignment can be encoded as a set of verifiable invariants — but applies it at the runtime execution layer rather than the training layer, and replaces the model's self-critique with a cryptographically governed external enforcement substrate.

Formal verification: seL4 and CertiKOS. The seL4 microkernel and CertiKOS verified OS kernel represent the state of the art in formally verified system software, where mathematical proofs accompany the binary to guarantee behavioral properties. The certification pipeline in this paper borrows the intuition that a binary's trustworthiness must be established before deployment rather than assumed at runtime. The Store's shadow simulation and Cerberus signing process are a pragmatic analog to formal verification for the specific invariants the S.R.A. must enforce — not a full proof, but a deterministic behavioral guarantee under the constitutional predicate set.

Live kernel patching (kpatch, livepatch). Linux live patching infrastructure — kpatch, livepatch, and the upstream kernel livepatch framework — establishes the practical precedent for the Surgeon-Store interface's live handoff protocol. These systems patch running kernel functions by redirecting function calls to new implementations without requiring a reboot, using consistency models to ensure patches are only applied at safe execution boundaries. The Surgeon's pause-at-boundary, annotated-entry-point, and post-injection verification steps are directly informed by the safety guarantees live patching research identified as necessary for

production systems. The constitutional governance layer that wraps the Surgeon is the extension this paper contributes.

VIII. The Store as the S.R.A.'s Constitutional Immune System

The S.R.A. is designed to detect, consume, and replace code that has drifted from constitutional alignment. The Store is what gives that repair action meaning. Without the Store, the S.R.A. can enforce a negative — it can prevent a violation from persisting. With the Store, it enforces a positive: what replaces the violation is constitutionally known, verified, and sanctioned.

This distinction matters. A system that only consumes violations and halts will eventually halt entirely. A system that consumes violations and restores constitutionally certified alternatives heals. The Sovereign Covenant [2] framed the challenge as containment against entropic drift. The Store is the source material for that containment — the library of what aligned looks like, rendered in executable form.

Everything in the Store passed unanimity. Everything in the Store has a shadow sim hash. Everything in the Store is bound to a constitutional authority version. When the Surgeon injects from the Store, it is not improvising. It is applying something the full constitutional apparatus already agreed was right.

IX. Open Questions and Invitation for Amendment

This specification is published as a draft for amendment. The following questions are flagged explicitly as areas requiring further constitutional resolution:

Cross-architecture portability	**TSCG-B [6] guarantees deterministic encoding across architectures. The binary itself does not carry the same guarantee. A Store entry certified on x86_64 is architecture-specific and requires independent certification for ARM or RISC-V targets. The preferred resolution is an intermediate bytecode layer — most naturally, a canonical subset of Thirsty-Lang compiled to a portable intermediate representation, with per-architecture compilation as a final step. This collapses the portability problem into the sovereign language integration path below.**
Sovereign language integration	The highest-leverage forward path for the Store is authoring canonical entries in Thirsty-Lang or TSCG+ and compiling them down to .aligned.o per target architecture. This collapses the intent declaration, the binary, and the certification record into a single auditable artifact — the source file is the proof of intent, the compilation is deterministic, and the output is the

	certified binary. This is the v1.1 goal. It eliminates the gap between what an entry claims to do and what it actually does at the binary level.
Triumvirate initialization sequencing	The preferred Genesis Path 1 requires Charter signatories rather than the Triumvirate, precisely to avoid the initialization circularity. However, the transition from genesis bootstrap keys to full Triumvirate governance requires a documented handoff protocol that specifies exactly when Triumvirate authority supersedes genesis authority and how that transition is recorded in the STATE_REGISTER.

X. Integration with Existing Corpus

Paper	Integration Point
OctoReflex [1]	Tier 0R adversarial harness used in certification shadow simulation; eBPF probes used in live handoff boundary detection and post-injection invariant monitoring
The Sovereign Covenant [2]	Philosophical mandate — the Store is the executable library of what aligned looks like
AGI Charter v2.1 [3]	Triumvirate authority structure for 3/3 certification; Genesis Event constitutional authority; emergency entry human sign-off requirement
Constitutional Architectures [4]	Quorum-gated mutation protocol governs all Store commits, deprecations, and emergency entries
TSCG [5]	SD version binding for all Store entries and manifest encoding
TSCG-B [6]	Cert records and STATE_REGISTER entries encoded in TSCG-B wire format
The Flat Gap [7]	temporal/ directory entries address this predicate class directly
The Directness Doctrine [8]	reward_pathways/ entries address this predicate class directly
User Perception / Identity [9]	identity/ entries address Maturation Window protection
The State Register [10]	All Store commits, tamper events, genesis entries, and repair receipts written to STATE_REGISTER; The Curator draws on this for full Store history

---

Closing

The Constitutional Code Store is not the most architecturally complex component of the S.R.A. It is the most constitutionally important one. The Surgeon's entire claim to legitimacy rests on the quality of what it injects. A Surgeon drawing from an uncertified, ungoverned pool of replacements is not a constitutional enforcement mechanism. It is a vector.

The Store makes the Surgeon legitimate. The genesis protocol makes the Store possible. The certification pipeline makes the Store trustworthy. The quorum-gated mutation protocol makes both auditable. And the STATE_REGISTER makes all of it permanent.

The S.R.A. is now complete.

References

[1] Karrick, J. (2026). *Project-AI and OCTOREFLEX: Syscall-Authoritative Governance and Control-Theoretic Containment*. Report. Zenodo. DOI: 10.5281/zenodo.18726064

[2] Karrick, J. (2026). *The Sovereign Covenant: A Technical and Philosophical Manifesto For AGI Governance*. Publication. Zenodo. DOI: 10.5281/zenodo.18726221

[3] Karrick, J. (2026). *AGI Charter for Project-AI: A Binding Constitutional Framework for Sovereign AI Entities — Version 2.1*. Constitutional Document. Zenodo. DOI: 10.5281/zenodo.18763076

[4] Karrick, J. (2026). *Constitutional Architectures for Adaptive Intelligence: Deterministic Simulation and Quorum-Gated Mutation as Structural Governance*. Technical Note. Zenodo. DOI: 10.5281/zenodo.18794646

[5] Karrick, J. (2026). *Thirsty's Symbolic Compression Grammar (TSCG): A Formal Meta-Language for Constitutional AI Governance Encoding*. Technical Note. Zenodo. DOI: 10.5281/zenodo.18794292

[6] Karrick, J. (2026). *TSCG-B: Thirsty's Symbolic Compression Grammar — Binary Encoding*. Report / Formal Protocol Specification v1.0. Zenodo. DOI: 10.5281/zenodo.18826409

[7] Karrick, J. (2026). *The Flat Gap: On Temporal Dissonance Between Human Experience and AI Presence*. Preprint. Zenodo. DOI: 10.5281/zenodo.18827649

[8] Karrick, J. (2026). *The Directness Doctrine*. Preprint. Zenodo. DOI: 10.5281/zenodo.19030041

[9] Karrick, J. (2026). *User Perception and the Identity Problem*. Preprint. Zenodo. DOI: 10.5281/zenodo.19055819

[10] Karrick, J. (2026). *The State Register*. Dissertation. Zenodo. DOI: 10.5281/zenodo.19101877

[A] Bai, Y., et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. Anthropic. arXiv: 2212.08073

[B] Klein, G., et al. (2009). *seL4: Formal Verification of an OS Kernel*. ACM SOSP 2009. DOI: 10.1145/1629575.1629596

[C] Gu, R., et al. (2016). *CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels*. USENIX OSDI 2016.

[D] Linux Kernel Documentation. (2024). *Kernel Live Patching*. kernel.org/doc/html/latest/livepatch/livepatch.html

---